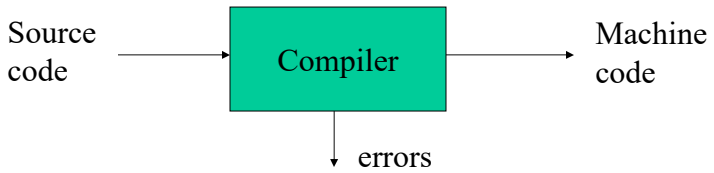
CS-411:Compiler Construction

1. Introduction

1

Compiler Construction

- Compiler Construction is an area of computer science that focuses on designing and implementing compilers, which translate high-level programming languages into machine code.



2

Compilers

- Compiler
  - A program that translates from 1 language to another
  - It must preserve semantics of the source
  - It should create an efficient version of the target language
- In the beginning, there was machine language
  - Ugly – writing code, debugging
  - Then came textual assembly
  - High-level languages – Fortran, Pascal, C, C++
  - Machine structures became too complex and software management too difficult to continue with low-level languages

3

Compiler learning

- Isn't it an old discipline?
  - Yes, it is a well-established discipline
  - Algorithms, methods and techniques are researched and developed in early stages of computer science growth
  - There are many compilers around and many tools to generate them automatically
- So, why we need to learn it?
  - Although you may never write a full compiler
  - But the techniques we learn is useful in many tasks like writing an interpreter for a scripting language, validation checking for forms and so on

4

Why Study Compilers? (1)

- Become a better programmer(!)
  - Insight into interaction between languages, compilers, and hardware
  - Understanding of implementation techniques
  - Better intuition about what your code does

5

Why Study Compilers? (2)

- Compiler techniques are everywhere
  - Parsing (little languages, interpreters, XML)
  - Database engines, query languages
  - AI: domain-specific languages
  - Natural Language Processing
  - Text processing
    - Tex/LaTex, Postscript, pdf
  - Hardware: model-checking tools

6

Why Study Compilers? (3)

- Fascinating blend of theory and engineering
  - Direct applications of theory to practice
    - Parsing, scanning, static analysis
  - Some very difficult problems (NP-hard or worse)
    - Resource allocation, “optimization”, etc.
    - Need to come up with good-enough approximations/heuristics

7

Why Study Compilers? (4)

- You might even write a compiler some day!
  - You’ll almost certainly write parsers and interpreters in some context if you haven’t already

8

Course Scope

- Aim:
  - To learn techniques of a modern compiler
- Books:
  - Des Watson, A Practical Approach to Compiler Construction, Springer, 2017.
  - Compilers – Principles, Techniques and Tools, Second Edition by Alfred V. Aho, Ravi Sethi, Jeffery D. Ullman

9

Prerequisites

- Problem Solving & Programming (C++, Python)
- Data Structures
- Algorithm Design
- Theory of Automata
- Computer Architecture
- Assembly Programming
- Operating Systems

10

Topics

- High Level Languages
- Lexical analysis (Scanning)
- Syntax Analysis (Parsing)
- Syntax Directed Translation
- Intermediate Code Generation
- Run-time Environments
- Code Generation
- Machine Independent Optimization

11

Grading policy

- Midterm (25 Marks)
- Final exam (50 Marks)
- Assignments (20 Marks)
- Quizes (5 Marks)

12

Terminology

- Compiler:
  - a program that translates an *executable* program in one language into an *executable* program in another language
  - we expect the program produced by the compiler to be better, in some way, than the original
- Interpreter:
  - a program that reads an *executable* program and produces the results of running that program
  - usually, this involves executing the source program in some fashion
- Our course is mainly about compilers but many of the same issues arise in interpreters

13

Common Issues

- Compilers and interpreters both must read the input – a stream of characters – and "understand" it; *analysis*

w h i l e (k < l e n g t h) { <nl> <tab> i f (a [k] > 0

) <nl> <tab> <tab>{ n P o s + + ; } <nl> <tab> }

14

Interpreter

- Interpreter
  - Execution engine
  - Program execution interleaved with analysis

```
running = true;
while (running) {
    analyze next statement;
    execute that statement;
}
```

  - Usually need repeated analysis of statements (particularly in loops, functions)
  - But: immediate execution, good debugging & interaction

15

Compiler

- Read and analyze entire program
- Translate to semantically equivalent program in another language
  - Easier to execute or more efficient
  - Should “improve” the program in some fashion

16

Typical Implementations

- Compilers
  - FORTRAN, C, C++, Java, COBOL, etc. etc.
  - Strong need for optimization in many cases
- Interpreters
  - PERL, Python, Ruby, awk, sed, shells, Scheme/Lisp/ML, postscript/pdf, Java VM
  - Particularly effective if interpreter overhead is low relative to execution cost of individual statements

17

Hybrid approaches

- Well-known example: Java
  - Compile Java source to byte codes – Java Virtual Machine language (.class files)
  - Execution
    - Interpret byte codes directly, or
    - Compile some or all byte codes to native code
      - Just-In-Time compiler (JIT) – detect hot spots & compile on the fly to native code – standard these days
- Variation: .NET
  - Compilers generate MSIL
  - All IL compiled to native code before execution

18

High-Level Languages

- In the 1940s it was clear that there was a need for software tools to support the programming process.
- Programming was done in *machine code*
  - required considerable skill and was hard work
- Assembly language
  - relieving the programmer from having to deal with much of the low-level detail
  - but require an *assembler*

19

High-Level Languages (Cont..)

- Development of high-level languages gathered speed in the 1950.
- Need for compilers and other tools for the implementation of these languages.
- The importance of formal language specifications was recognized
- The correspondence between particular grammar types and implementation was understood

20

High-Level Languages (Cont..)

- The extensive use of high-level languages prompted the rapid development of a wide range of new languages.
  - COBOL for business applications
  - FORTRAN for numerical computation
  - PL/I general-purpose

21

Advantages of High-Level Languages

- Problem solving is significantly faster
- High-level language programs
  - Easier to read
  - Understand
  - Maintain
- High-level languages are easier to learn.
- Programs are structured more easily
- Data structuring features

22

Advantages of High-Level Languages (Cont..)

- Object orientation
- Support for asynchronous processes and parallelism
- Software portability
  - Machine independence
  - Java

23

Advantages of High-Level Languages (Cont..)

- Compile-time checking can remove many bugs at an early stage
  - variable declarations
  - type checking
  - variables initialization
  - compatibility in function arguments
- The compiler can insert runtime code such as array bound checking

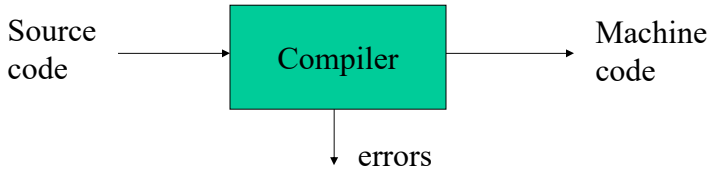
24

Disadvantages of High-Level Languages

- The program may need to perform some low-level, hardware-specific operations which do not correspond to a high-level language feature.
- In most high-level languages there is no way to express direct machine addressing
- Less efficiency in terms of execution speed

25

Abstract view

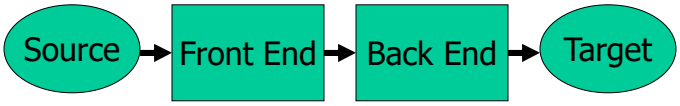


- Recognizes legal (and illegal) programs
- Generate correct code
- Manage storage of all variables and code
- Agreement on format for object (or assembly) code

26

Structure of a Compiler

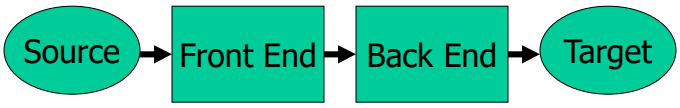
- First approximation
  - Front end: analysis
    - Read source program and understand its structure and meaning
  - Back end: synthesis
    - Generate equivalent target language program



27

Implications

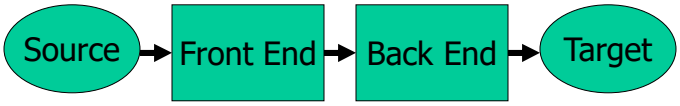
- Must recognize legal programs (& complain about illegal ones)
- Must generate correct code
- Must manage storage of all variables/data
- Must agree with OS & linker on target format



28

More Implications

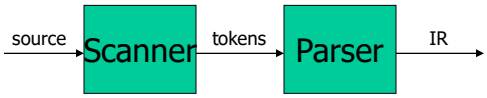
- Need some sort of Intermediate Representation(s) (IR)
- Front end maps source into IR
- Back end maps IR to target machine code
- Often multiple IRs – higher level at first, lower level in later phases



29

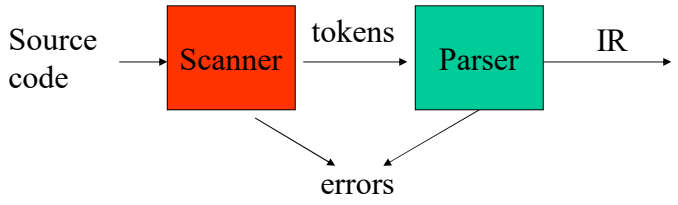
Front End

- Split into two parts
  - Scanner: Responsible for converting character stream to token stream
    - Also strips out white space, comments
  - Parser: Reads token stream; generates IR
- Both of these can be generated automatically
  - Source language specified by a formal grammar
  - Tools read the grammar and generate scanner & parser (either table-driven or hard-coded)



30

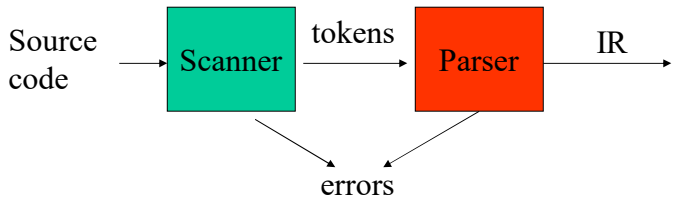
Front end



- Scanner:
  - Maps characters into tokens – the basic unit of syntax
    - x = x + y becomes <id, x> = <id, x> + <id, y>
  - Typical tokens: number, id, +, -, *, /, do, end
  - Eliminate white space (tabs, blanks, comments)
- A key issue is speed so instead of using a tool like LEX it sometimes needed to write your own scanner

31

Front end



- Parser:
  - Recognize context-free syntax
  - Guide context-sensitive analysis
  - Construct IR
  - Produce meaningful error messages
  - Attempt error correction
- There are parser generators like YACC which automates much of the work

32

Front end

- Context free grammars are used to represent programming language syntaxes:

<expr> ::= <expr> <op> <term> | <term>

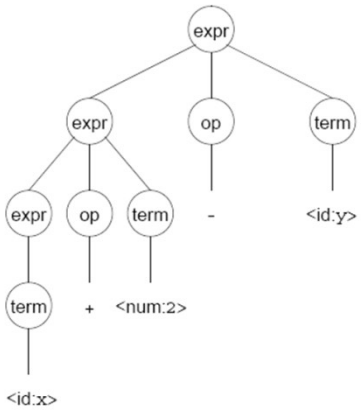
<term> ::= <number> | <id>

<op> ::= + | -

33

Front end

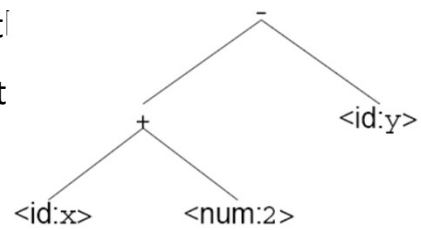
- A parser tries to map a program to the syntactic elements defined in the grammar
- A parse can be represented by a tree called a parse or syntax tree
- x+2-y



34

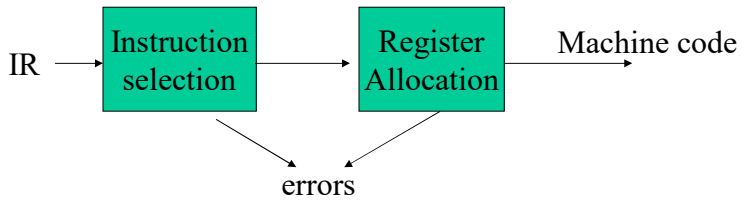
Front end

- A parse tree can be represented more compact referred to as Abstract Synt Tree (AST)
- AST is often used as IR between front end and back end



35

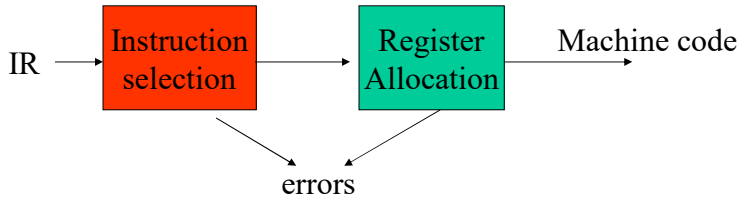
Back end



- Translate IR into target machine code
- Choose instructions for each IR operation
- Decide what to keep in registers at each point
- Ensure conformance with system interfaces

36

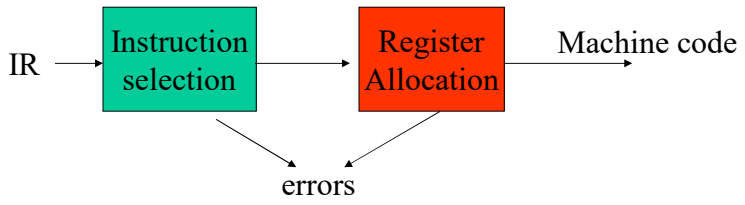
Back end



- Produce compact fast code
- Use available addressing modes

37

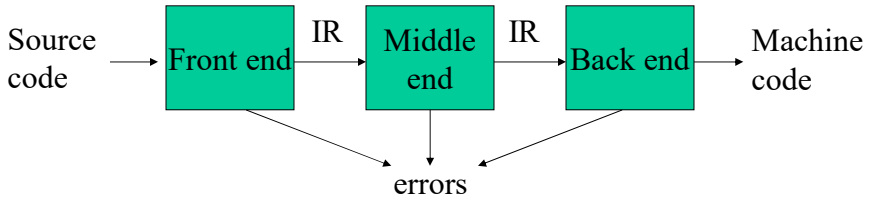
Back end



- Have a value in a register when used
- Limited resources
- Optimal allocation is difficult

38

Traditional three pass compiler



- Code improvement analyzes and change IR
- Goal is to reduce runtime

39

Middle end (optimizer)

- Modern optimizers are usually built as a set of passes
- Typical passes
  - Constant propagation
  - Common sub-expression elimination
  - Redundant store elimination
  - Dead code elimination

40

Defining a Programming Language

- Important to users of the programming language as well as to the compiler writer.
- how to write programs in the language
- *Syntax*
  - Defines the sequences of characters that could form a valid program
- *Semantics*
  - the meaning of these programs

41

Defining a Programming Language

- The language definition should be
  - Clear
  - Precise
  - Complete
  - Unambiguous
  - Understandable
- Programming languages have been defined using natural language and ambiguities
- syntax is usually done using a more formal approach

42

Backus–Naur Form (BNF)

- Metalanguage
- Its use in the definition of the syntax of ALGOL 60
- It is a very simple
- powerful metalanguage
- used extensively to support the formal definitions of a huge range of languages

43

Backus–Naur Form (BNF)

- BNF specification consists of a set of rules
- Rule defining a symbol
  - *non-terminal symbol*s
    - in angle brackets in BNF
  - *terminal symbol*
    - cannot be expanded further
    - Tokens
  - A Start Symbol
    - *non-terminal*

44

A trivial language

- <sentence> ::= <subject> <verb> <object>
- <subject> ::= <article> <noun>
- <object> ::= <article> <noun> / <article> <adjective> <noun>
- <verb> ::= watches / hears / likes
- <article> ::= a / the
- <noun> ::= man / woman / bicycle / book
- <adjective> ::= red / big / beautiful

45

Example

- <sentence>
- <subject><verb><object>
- <article><noun><verb><object>
- the <noun><verb><object>
- the woman <verb><object>
- the woman watches <object>
- the woman watches <article><adjective><noun>
- the woman watches a <adjective><noun>
- the woman watches a beautiful <noun>
- the woman watches a beautiful bicycle

46

BNF for Expression

- Slightly more complicated example
  - <expr> ::= <term> / <expr> + <term> / <expr> - <term>
  - <term> ::= <factor> / <term> * <factor> / <term> / <factor>
  - <factor> ::= <integer> / (<expr>)
  - <integer> ::= <digit> / <integer> <digit>
  - <digit> ::= 0 / 1 / 2 / 3 / 4 / 5 / 6 / 7 / 8 / 9
  - Expressions such as 2+3+4/5 and 2+3*(44-567)

47

BNF

- There is no upper limit to the length of expressions that can be generated by these particular rules.
- This is because these rules make use of *recursion*
- Consider the generation or *derivation* of the expression 1+2*3

48

BNF

- <expr>
- <expr> + <term>
- <term> + <term>
- <factor> + <term>
- 1 + <term>
- 1 + <term> * <factor>
- 1 + <factor> * <factor>
- 1 + 2 * <factor>
- 1 + 2 * 3

49

BNF

- Note particularly that in the expansion
  - 1+2*3
  - 1+ <term>
  - 2*3 grouped as a single entity called a <term>
- BNF rules can be used to support the idea of *operator precedence*
- * operator is higher than the precedence of the +
- The parsing of the BNF rules allow the specification of the precedence of operators.

50

BNF

- BNF rules can be used to express the *associativity* of operators.
- consider the generation 1+2+3.
- Here, the 1+2 part of the expression is grouped as a single entity called a <term>.
- Therefore, the expression 1+2+3 is interpreted as (1+2)+3.
- The + operator is *left-associative*.

51

BNF

- If different associativity or precedence rules are required, then the BNF could be modified to express these different rules.
- It is perhaps surprising that such a simple metalanguage can do all this.

52

Extended Backus–Naur Form (EBNF)

- There are several different variants of the low-level syntax of BNF-like metalanguages, but one variant became popular after its use in the ISO Pascal Standard.
- This variant was called *Extended Backus–Naur Form* (*EBNF*)
- It retains the basic principles of BNF but the syntactic detail is a little different.

53

Extended Backus–Naur Form (EBNF)

- *expr* = *term* | *expr* "+" *term* | *expr* "-" *term*.
- *term* = *factor* | *term* "*" *factor* | *term* "/" *factor*.
- *factor* = *integer* | "("*expr*")".
- *integer* = *digit* | *integer digit*.
- *digit* = "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9".

54

Extended Backus–Naur Form (EBNF)

- Some other key features.
  - Parentheses can be used to indicate grouping in the rule.
  - There is a specific feature to indicate optionality in a rule: [X] specifies zero or one instance of X, in other words specifying that X is optional.
  - Repetition (not by recursion) is supported too: {X} implies zero or more instances of X.

55

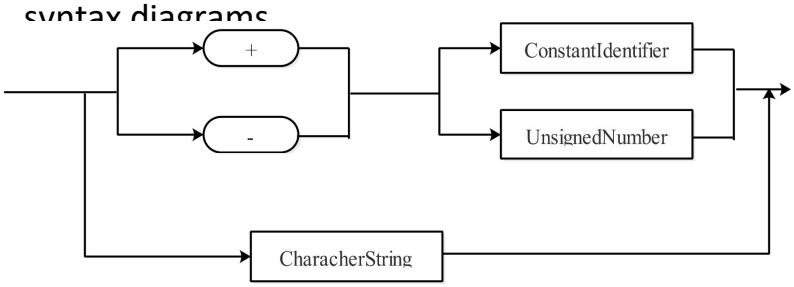
Extended Backus–Naur Form (EBNF)

- We can therefore write an essentially equivalent set of rules:
- *expr* = *term* | *expr* ("+" | "-") *term*.
- *term* = *factor* | *term* ("*" | "/") *factor*.
- *factor* = *integer* | "("*expr*")".
- *integer* = *digit* {*digit*}.
- *digit* = "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9".

56

Syntax Diagram

- Graphical representation of Syntax
- Complete syntax definition is made up of a set of syntax diagrams



57

Grammars

- The term “grammar” has a wide range of definitions
- We need a tight and formal definition for it when used in the context of programming languages.
- The idea of a set of BNF rules to represent the grammar of a language has already been discussed, but formally a little more is required.

58

Grammars

- The *grammar* (G) of a language is defined as a 4-tuple $G = (N, T, S, P)$ where:
  - N is the finite set of non-terminal symbols.
  - T is the finite set of terminal symbols (N and T are disjoint.)
  - S is the *starting symbol*, $S \in N$. The starting symbol is the unique non-terminal symbol that is used as the starting point for the generation of all the strings of the language.

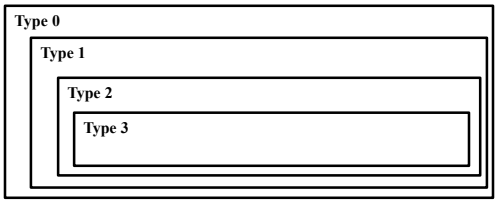
59

Grammars

- P is the finite set of *production rules*
  - A production rule defines a string transformation and it has the general form
    - $\alpha \rightarrow \beta$.
  - This rule specifies that any occurrence of the string α in the string to be transformed is replaced by the string β.
- There are constraints on the constitution of the strings α and β.
  - If U is defined by $U = N \cup T$
    - U is the set of all non-terminal and terminal symbols
    - then α has to be a member of the set of all non-empty strings that can be formed by the concatenation of members of U, and it has to contain at least one member of N.
    - β has to be a member of the set of all strings that can be formed by the concatenation of members of U, including the empty string
      - $\beta \in U*$

60

Chomsky Hierarchy



61

Chomsky Hierarchy

- *Chomsky type 0*
  - a *free grammar* or an *unrestricted grammar*
  - $\alpha \rightarrow \beta$
- *Chomsky type 1*
  - a *context-sensitive grammar*
  - $\alpha A \beta \rightarrow \alpha \gamma \beta$

62

Chomsky Hierarchy

- *Chomsky type 2*
  - a *context-free grammar*
  - $A \rightarrow \gamma$
  - where A is a single non-terminal symbol.
  - These productions correspond directly to BNF rules.
- *Chomsky type 3*
  - a *regular grammar* or a *finite-state grammar*
  - $A \rightarrow a$ or $A \rightarrow aB$

63